



Terraform State Management and Backend

Introduction

- ❖ Terraform uses a state file to keep track of infrastructure resources. Proper state management is crucial for consistency, collaboration, and security. This document covers Terraform state management and backend configurations.



What is Terraform State?

- ❖ Terraform state is a JSON file (`terraform.tfstate`) that records information about managed infrastructure. It enables Terraform to map resources in your configuration to real-world resources.



Benefits of Terraform State

- Tracks infrastructure changes
- Enables efficient plan and apply operations
- Helps resolve dependencies
- Supports collaboration in a team environment



Terraform Backend

- ❖ A Terraform backend determines how state is stored and accessed. By default, Terraform uses a local backend (`terraform.tfstate` stored locally), but remote backends offer better security, collaboration, and resilience.

◆ Types of Backends

Terraform supports multiple backends, including:

- **Local Backend:** Stores state in local disk (`terraform.tfstate`).
- **Remote Backend:** Stores state in cloud services for security and accessibility.



Secure Remote Backend Options



Cloud Storage

- AWS S3 (with DynamoDB for state locking)
- Azure Blob Storage
- Google Cloud Storage (GCS)
- Terraform Cloud & Terraform Enterprise



Database Backend

- Consul
- PostgreSQL



Configuring Remote Backends

Below is an example of configuring an AWS S3 backend with state locking using DynamoDB:

```
terraform
backend "s3" {
  bucket      = "my-terraform-state-bucket"
  key         = "state/terraform.tfstate"
  region      = "us-east-1"
  encrypt     = true
  dynamodb_table = "terraform-state-lock"
}
```



Best Practices for State Management

- Use Remote Backends: Store state remotely for security and collaboration.
- Enable State Locking: Prevents simultaneous modifications.
- Encrypt State Files: Use encryption at rest and in transit.
- Restrict Access: Use IAM policies to limit state access.
- Enable Versioning: Keep backups to prevent accidental loss.
- Avoid Storing Secrets: Use environment variables or secret managers.

Terraform State Management

Introduction

📌 Terraform state management is a critical aspect of Infrastructure as Code (IaC) that ensures the tracking, storage, and modification of deployed resources. Terraform maintains a state file (`terraform.tfstate`) to map real-world resources to configuration files, enabling efficient updates and resource management. 🔄💾📁

Terraform State Commands

1. Listing State Resources

📖 To view all managed resources in the Terraform state, use:

```
terraform state list
```

Example Output:

```
PS C:\Users\mc882\Downloads\terraform> terraform state list
aws_default_vpc.default
aws_instance.my-instance["tws-junoon-micro2"]
aws_instance.my-instance["tws-junoon-micro1"]
aws_key_pair.deployer
aws_security_group.my_security_group
```

2. Showing Detailed Resource Information

🔍 To get detailed information about a specific resource in the state:

```
terraform state show <resource_name>
```

Example:

```
PS C:\Users\mc882\Downloads\terraform> terraform state show
aws_key_pair.deployer
# aws_key_pair.deployer:
resource "aws_key_pair" "deployer" {
  arn          =
  "arn:aws:ec2:eu-west-1:686255949648:key-pair/terra-key-ec2"
  fingerprint  = "xxxxxxxxxxxxxx"
  id           = "terra-key-ec2"
  key_name     = "terra-key-ec2"
  key_name_prefix = null
  key_pair_id  = "key-0793616ac7c7ae972"
  key_type     = "ed25519"
  public_key   = "xxxxxxxxxx"
  tags_all     = {}
}
```

3. Removing a Resource from State

🚫 To remove a resource from the state without deleting it from the infrastructure:

```
terraform state rm <resource_name>
```

Example:

```
PS C:\Users\mc882\Downloads\terraform> terraform state rm
aws_key_pair.deployer
Removed aws_key_pair.deployer
Successfully removed 1 resource instance(s).
```

Output:

- ✓ Removed aws_key_pair.deployer
- ✓ Successfully removed 1 resource instance(s).

4. Importing an Existing Resource into State



To import an existing resource into Terraform state:

```
terraform import <resource_name> <resource_id>
```

Example:

```
terraform import aws_key_pair.deployer key-0793616ac7c7ae972
```

Output:

```
PS C:\Users\mc882\Downloads\terraform> terraform import  
aws_key_pair.deployer key-0793616ac7c7ae972  
aws_key_pair.deployer: Importing from ID "key-0793616ac7c7ae972"...  
aws_key_pair.deployer: Import prepared!  
Prepared aws_key_pair for import
```

Another example for an EC2 instance:

```
terraform import aws_instance.my_new_instance i-0f856b753dbd22b1
```

Output:

```
PS C:\Users\mc882\Downloads\terraform> terraform import aws_instanc  
e.my_new_instance i-0f856b753dbd22b1  
aws_instance.my_new_instance:porting from ID "i-0f856b753d1bd22b1".  
aws_instance.my_new_instance: Import prepared!  
Prepared aws_instance for import  
aws_instance.my_new_instance: Refreshing state...  
[id=i-0f856b753d1bd22b1]  
Import successful!  
The resources that were imported are shown above. These resources are  
now in  
your Terraform state and will henceforth be managed by Terraform.
```

5. Filtering Listed Resources

 To list specific resources in the Terraform state:

```
terraform state list <resource_name>
```

Example:

```
terraform state list aws_default_vpc.default
```

Output:

```
PS C:\Users\mc882\Downloads\terraform> terraform state list  
aws_default_vpc.default  
aws_instance.my_instance ["twS-junoon-automate-medium"]  
aws_instance.my_instance ["twS-junoon-automate-micro"]  
aws_instance.my_new_instance  
aws_security_group.my_security_group
```

Conclusion

 Terraform state management is essential for tracking and maintaining infrastructure resources efficiently. Using commands like `terraform state list`, `terraform state show`, `terraform state rm`, and `terraform import`, users can manage infrastructure with better control and visibility. Proper state management practices help maintain an accurate representation of deployed infrastructure, preventing drift and ensuring consistency in deployments.   



Secure Terraform State Management

- ❖ Terraform state files (`terraform.tfstate`) contain sensitive information like **access keys**, **secrets**, and **infrastructure details**, so securing them is crucial.

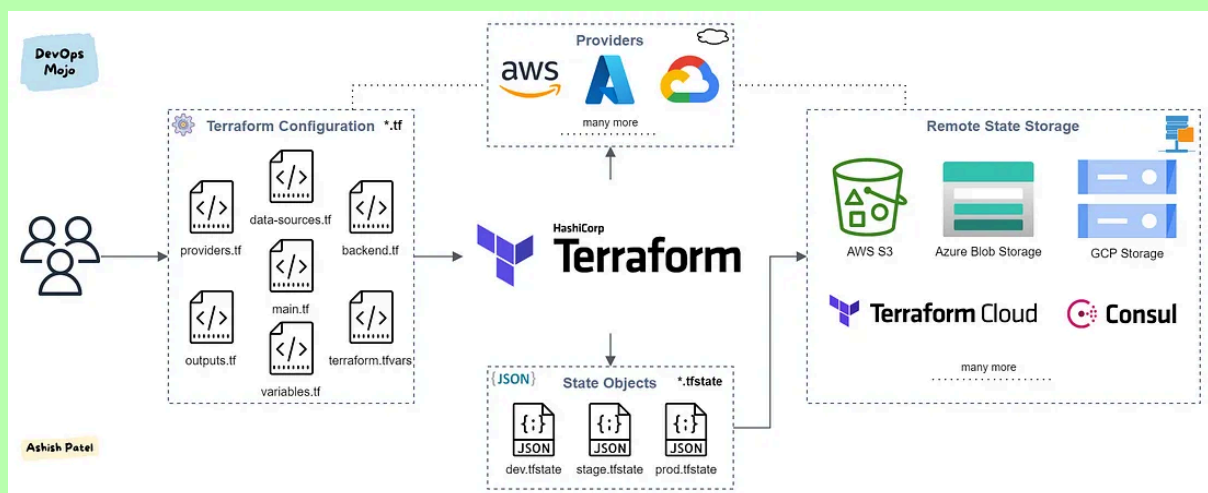
1 Store State Remotely (Avoid Local State)

By default, Terraform stores state locally in `terraform.tfstate`. This is insecure if multiple people are working on the infrastructure. Use **remote state storage** instead.



Best Remote Backends

- Amazon S3 + DynamoDB (for state locking)
- Terraform Cloud/Enterprise
- Google Cloud Storage (GCS)
- Azure Blob Storage
- HashiCorp Consul



◆ Example: AWS S3 + DynamoDB for Locking

📌 Step 1: Create an S3 Bucket

```
aws s3 mb s3://my-terraform-state-bucket
```

📌 Step 2: Create a DynamoDB Table for Locking

```
aws dynamodb create-table --table-name terraform-lock \  
  --attribute-definitions AttributeName=LockID,AttributeType=S \  
  --key-schema AttributeName=LockID,KeyType=HASH \  
  --billing-mode PAY_PER_REQUEST
```

📌 Step 3: Configure Backend in Terraform

Add this to your `backend.tf` file:

```
terraform {  
  backend "s3" {  
    bucket      = "my-terraform-state-bucket"  
    key         = "terraform.tfstate"  
    region      = "us-east-1"  
    encrypt     = true  
    dynamodb_table = "terraform-lock" # Enables state locking  
  }  
}
```

📌 Step 4: Initialize Backend

```
PS C:\Users\mc882\Downloads\remote-infra> terraform init  
Initializing the backend...  
  
Successfully configured the backend "s3"! Terraform will automatically  
use this backend unless the backend configuration changes.  
Initializing provider plugins...  
- Reusing previous version of hashicorp/aws from the dependency lock  
file  
- Using previously-installed hashicorp/aws v5.92.0  
  
Terraform has been successfully initialized!  
  
You may now begin working with Terraform. Try running "terraform plan"  
to see  
any changes that are required for your infrastructure. All Terraform
```

```
commands  
should now work.
```

If you ever set or change modules or backend configuration for Terraform, rerun this command to reinitialize your working directory. If you forget, other commands will detect it and remind you to do so if necessary.

Step 5: Plan Backend

```
PS C:\Users\mc882\Downloads\remote-infra> terraform plan
```

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:

- + create

Terraform planned the following actions, but **then** encountered a problem:

```
# aws_s3_bucket.remote_s3 will be created  
+ resource "aws_s3_bucket" "remote_s3" {  
  + acceleration_status      = (known after apply)  
  + acl                      = (known after apply)  
  + arn                     = (known after apply)  
  + bucket                  = "twS-junoon-state-bucket"  
  + bucket_domain_name      = (known after apply)  
  + bucket_prefix           = (known after apply)  
  + bucket_regional_domain_name = (known after apply)  
  + force_destroy           = false  
  + hosted_zone_id          = (known after apply)  
  + id                     = (known after apply)  
  + object_lock_enabled     = (known after apply)  
  + policy                  = (known after apply)  
  + region                  = (known after apply)  
  + request_payer           = (known after apply)  
  + tags                    = {  
    + "Name" = "twS-junoon-state-bucket"  
  }  
  + tags_all                = {  
    + "Name" = "twS-junoon-state-bucket"  
  }  
  + website_domain          = (known after apply)
```

```

+ website_endpoint           = (known after apply)

+ cors_rule (known after apply)

+ grant (known after apply)

+ lifecycle_rule (known after apply)

+ logging (known after apply)

+ object_lock_configuration (known after apply)

+ replication_configuration (known after apply)

+ server_side_encryption_configuration (known after apply)

+ versioning (known after apply)

+ website (known after apply)
}

```

Plan: 1 to add, 0 to change, 0 to destroy.

Step 6: Apply Backend

```

PS C:\Users\mc882\Downloads\remote-infra> terraform apply
aws_dynamodb_table.basic-dynamodb-table: Refreshing state...
[id=twj-junoon-state-table]

```

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:

+ create

Terraform will perform the following actions:

```

# aws_s3_bucket.remote_s3 will be created
+ resource "aws_s3_bucket" "remote_s3" {
  + acceleration_status = (known after apply)
  + acl                 = (known after apply)
  + arn                 = (known after apply)
  + bucket              = "twj-junoon-state-bucket12c"
  + bucket_domain_name = (known after apply)
  + bucket_prefix       = (known after apply)
}

```

```
+ bucket_regional_domain_name = (known after apply)
+ force_destroy                = false
+ hosted_zone_id               = (known after apply)
+ id                           = (known after apply)
+ object_lock_enabled           = (known after apply)
+ policy                        = (known after apply)
+ region                        = (known after apply)
+ request_payer                 = (known after apply)
+ tags                          = {
  + "Name" = "twS-junoon-state-bucket"
}
+ tags_all                      = {
  + "Name" = "twS-junoon-state-bucket"
}
+ website_domain                = (known after apply)
+ website_endpoint              = (known after apply)

+ cors_rule (known after apply)

+ grant (known after apply)

+ lifecycle_rule (known after apply)

+ logging (known after apply)

+ object_lock_configuration (known after apply)

+ replication_configuration (known after apply)

+ server_side_encryption_configuration (known after apply)

+ versioning (known after apply)

+ website (known after apply)
}
```

Plan: 1 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?

Terraform will perform the actions described above.

Only 'yes' will be accepted to approve.

Enter a value: yes

aws_s3_bucket.remote_s3: Creating...

aws_s3_bucket.remote_s3: Creation complete after 7s

```
[id=twS-junoon-state-bucket]
```

```
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
```

Now, Terraform state is securely stored in S3, with **state locking** enabled via DynamoDB.

2 Encrypt State Files

If you're storing the state locally (not recommended), **encrypt** it:

```
gpg --symmetric --cipher-algo AES256 terraform.tfstate
```

To decrypt:

```
gpg --decrypt terraform.tfstate.gpg > terraform.tfstate
```

3 Restrict Access to State Files

- **S3 Bucket:** Use **IAM policies** to limit access.
- **Terraform Cloud:** Only allow authenticated users.

Local State: Restrict file permissions.

```
chmod 600 terraform.tfstate
```

4 Enable Role-Based Access Control (RBAC)

- Use **AWS IAM Roles** or **Terraform Cloud RBAC** to control who can access

and modify the state.

Example IAM policy for **read-only state access**:

```
{
  "Effect": "Allow",
  "Action": ["s3:GetObject"],
  "Resource": "arn:aws:s3:::my-terraform-state-bucket/terraform.tfstate"
}
```

5 Use **terraform state pull/push** (Avoid Direct Editing)

Pull the latest state:

```
terraform state pull > terraform.tfstate
```

Push an updated state manually (only if needed):

```
terraform state push terraform.tfstate
```



Summary

Security Measure	Action
Use Remote Backend	Store state in S3/GCS/Azure/Terraform Cloud
Enable State Locking	Use DynamoDB (AWS) to prevent conflicts
Encrypt State	Use GPG for local state encryption
Restrict Access	Apply IAM Policies, RBAC, or File Permissions
Avoid Direct State Edits	Use terraform state pull/push cautiously

By implementing these steps, your Terraform state will be **secure**, **encrypted**, and **protected** from unauthorized access. 🚀🔒

Conclusion

Managing Terraform state effectively ensures consistency, security, and efficient infrastructure management. By leveraging remote backends, encryption, and access controls, teams can safely collaborate and maintain their infrastructure state.

References

- [Terraform Documentation](#)
- [Terraform Backend Configuration](#)



Remote State Locking

- ❖ When using remote state storage, Terraform implements **state locking** to prevent multiple users from making conflicting changes at the same time. State locking ensures that only one Terraform process can modify the state, reducing the risk of corruption or conflicts.

1 Why Use State Locking?

- ✓ Prevents concurrent state modifications
- ✓ Ensures consistency across teams
- ✓ Reduces risk of state corruption

2 How State Locking Works?

- ❖ Terraform acquires a lock on the state file before making changes and releases it after the operation is complete. If another process tries to modify the state while it's locked, Terraform prevents it and displays a lock error.

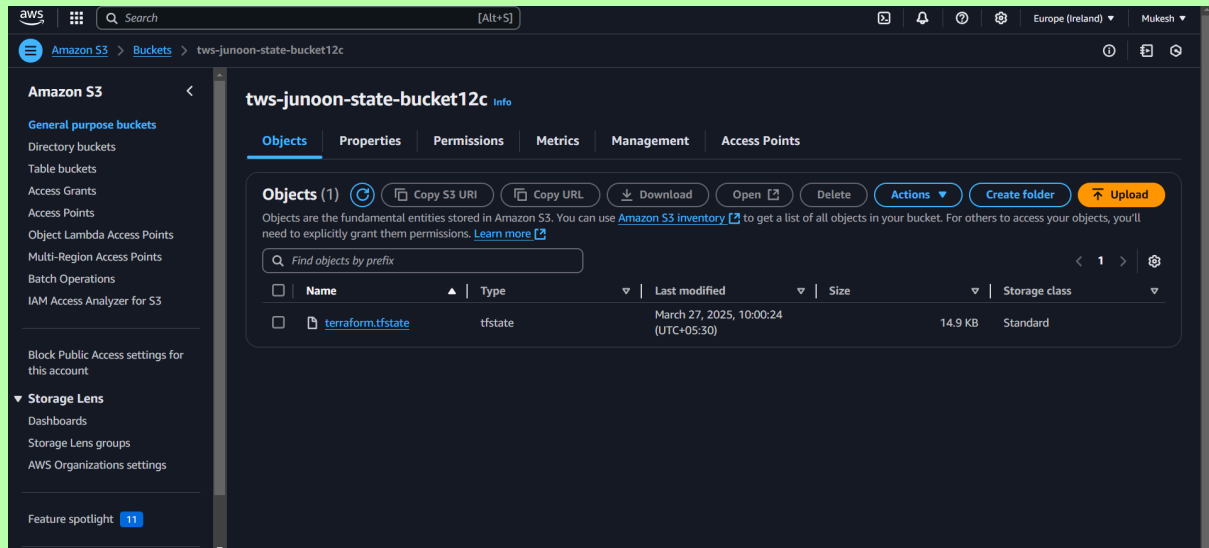
3 Backends That Support State Locking

- ♦ AWS S3 with DynamoDB
- ♦ Azure Blob Storage
- ♦ Google Cloud Storage
- ♦ HashiCorp Consul

4 Example: Enabling Remote State Locking on AWS S3

To enable **state locking** in an S3 backend, use **DynamoDB** as the lock table:

```
terraform {
  backend "s3" {
    bucket      = "my-terraform-state"
    key         = "terraform.tfstate"
    region      = "us-east-1"
    dynamodb_table = "terraform-lock"
    encrypt     = true
  }
}
```

5 Forcing Unlock (Use with Caution!)

- ❖ If Terraform fails to release a lock (e.g., due to an interrupted process), you can manually unlock it using:

```
terraform force-unlock <LOCK_ID>
```

Example:

```
terraform force-unlock 12345678-90ab-cdef-1234-567890abcdef
```

⚠ **Caution:** Forcing unlocks can cause state corruption if another Terraform process is modifying the state.
